Understanding smolVLA: Asynchronous Inference

Control Loops, Queue Management, and Policy Servers

1 Introduction

Modern visuomotor policies, including **smolVLA**, do not simply output one action per timestep. Instead, they produce **action chunks**—sequences of n low-level commands queued for execution. While this formulation is powerful, it raises an important systems question: *when and how should the next action chunk be predicted?*

Section 3.3 of the smolVLA paper introduces an **Asynchronous Inference** stack designed to decouple the physical execution of actions from the computational burden of predicting them. This document details the challenges of standard synchronous inference and how the asynchronous approach solves them.

2 The Inference Bottleneck

To understand the need for an asynchronous stack, we must examine the limitations of traditional inference strategies when deploying VLA policies to physical robots.

Let the predicted chunk at time t be $A_t = (a_t, a_{t+1}, \ldots, a_{t+n})$.

2.1 Synchronous (Open-Loop) Inference

In a purely synchronous approach, the robot executes the entire chunk A_t (exhausting all n steps) before capturing a new observation o_{t+n} and sending it to the policy.

- **Advantage:** It minimizes the average computational load because inference only runs once every n timesteps.
- **Disadvantage:** It introduces "blind lags." When the chunk runs out, the physical robot must wait idle while the policy computes the next chunk. It is also completely blind to unexpected environmental changes during the n-step open-loop execution.

2.2 Continuous Timestep Inference

At the opposite extreme, the robot can compute a new chunk at *every* timestep t, acting continuously and aggregating overlapping chunks.

- **Advantage:** Highly adaptive and reactive.
- **Disadvantage:** Prohibitively expensive, making it completely unviable for resource-constrained scenarios (like onboard edge deployments on low-power robots).

3 The Asynchronous Inference Stack

To bridge the gap between complete open-loop execution and expensive continuous prediction, the authors propose an **asynchronous (async) inference** methodology.

This architecture fundamentally splits the system into two concurrent entities:

1. **RobotClient:** A lightweight process running on the robot or an edge device. It is solely responsible for popping actions off the local queue and sending them to the motors.

2. **PolicyServer:** A heavily provisioned process (often running on a remote machine with powerful GPUs). It receives observations, runs the VLA inference, and returns populated action chunks.

3.1 The Triggering Mechanism

Instead of waiting for the action queue to empty, the `RobotClient` continuously monitors the remaining length of the queue. The client initiates a new inference request as soon as the queue capacity drops below a fractional threshold $g \in [0, 1]$:

$$\frac{|A_t|}{n} < g$$

When this condition is true, the client captures a fresh observation o_{t+1} and sends it to the `PolicyServer` in a non-blocking background thread. Meanwhile, the main control loop continues happily executing the remaining actions in the current queue.

3.2 Queue Aggregation

When the new chunk $\tilde{A}_{t+1}$ eventually arrives from the server, the client will likely still have unexecuted actions in its old queue. The system smoothly aggregates the overlapping timesteps, effectively updating the robot's immediate future with the newest perceptual information.

4 Analytical Properties and the Core Trade-Offs

4.1 The Role of the Threshold g

The parameter g effectively controls the trade-off between strict sequential wait times and continuous computation:

- **$g = 0$ (Sequential Limit):** Behaves like fully synchronous deployment. The robot is left idle during the inference round-trip latency. It's cheap but jerky.
- **$g = 0.7$ (Balanced Asynchronous):** The client starts asking for new actions when 30% of the chunk has been exhausted. This gives the server enough time to process and return new actions before the queue empties.
- **$g = 1$ (Compute-Intensive Limit):** Triggers an observation sent at the slightest drop in the queue. Maximally reactive but highly consumptive.

To avoid an exhausted queue altogether, assuming an expected inference round-trip latency $E[\ell]$ and a hardware control cycle time Δt (e.g., 33ms for 30Hz), one must set the threshold such that:

$$g \geq \frac{E[\ell]}{n \cdot \Delta t}$$

4.2 The Observation Similarity Filter

Because the `RobotClient` runs faster than the network, setting $g = 0.7$ might cause the client to request inference multiple times in quick succession before the first request even returns.

To prevent this flooding, the implementation includes a **similarity filter**. Before sending a new frame o_{t+1} to the server, it compares it against the previously forwarded observation in the *joint-space* (proprioception).

- If the absolute difference is beneath a threshold ϵ, the observation is deemed a "near-duplicate" and is discarded without making a server request.
- **Failsafe:** If the action queue actually empties completely, the similarity filter is overridden, forcing the system to retrieve new actions no matter what.

5 A Step-by-Step Walkthrough: One Full Pass

To clarify the mechanics of the threshold g, the similarity filter, and queue aggregation, let's trace a practical example. Assume the action chunk size is $n = 50$, and our threshold is $g = 0.7$.

5.1 1. Initial Startup ($t = 0$)

- The `RobotClient` starts with an empty queue. Because the queue is empty, the similarity filter is completely bypassed.
- It captures an observation o_0 and sends it to the `PolicyServer`.
- The client physically waits until the server replies with the first chunk of 50 actions.

5.2 2. Execution and Triggering

- The client begins executing the 50 actions, popping one per control cycle.
- The async trigger condition is $\frac{|A_t|}{n} < g$. With $g = 0.7$ and $n = 50$, the trigger fires when the remaining queue drops below **35 actions**.
- Thus, after executing exactly **16 actions**, the queue has 34 actions left. The condition is met!

5.3 3. The Similarity Filter (What happens if it's too close?)

At this exact moment, the client captures a new observation o_{new} and compares it to o_{last} (the one it sent at $t = 0$).

- **Case A: Observations are too similar.** If the robot hasn't moved much (e.g., it's moving slowly), the similarity filter flags it as a "near-duplicate". The client drops this observation and sends **nothing** to the server. The client simply executes the next action (now 33 left) and checks again on the next tick.
- **The Degenerate Case:** If the robot continues to move too little (or if current movement is perceived as too similar to the reference frame), it will continuously drop frames until the queue hits 0. Once the queue is fully empty ($|A_t| = 0$), the similarity filter is forcefully bypassed, and it is guaranteed to send a frame to the server.
- **Result of Case A:** In this edge case, the system effectively "falls back" to **Synchronous behavior**. The robot will be momentarily idle while it waits for the server response, exactly as if g were set to 0. This ensures that even if the filter is too aggressive, the robot never stays frozen forever.
- **Case B: Observations are different enough.** The client sends o_{new} to the `PolicyServer` to request a new chunk.

5.4 4. Round-Trip Latency and Queue Aggregation

Let's assume **Case B** happened. The server is now computing the new 50-step chunk.

- While the server computes, the client is **not** blocked. It continues happily popping actions from the remaining 34 steps.
- Suppose the network trip and GPU inference take 5 control ticks. By the time the client receives the new chunk from the server, its old queue has $34 - 5 = \mathbf{29}$ **actions** remaining.
- **Aggregation:** The newly received chunk has 50 actions. The first 29 actions of this new chunk overlap in time with the 29 remaining actions in the old queue. The algorithm "aggregates" these overlaps (e.g. by averaging them or applying temporal ensembling to smooth the trajectory). The remaining 21 actions from the new chunk are then appended to the end, resulting in a freshly topped-up queue of exactly 50 actions.
- The process then repeats.

6 Synchronization and Weighting Details

6.1 How does the Robot know the latency?

A common question is: *How does the RobotClient know which steps from the new chunk to align with its current physical position?*

The answer is **Step-Index Tracking**:

1. When the `RobotClient` captures o_{new}, it marks its current internal step counter (e.g., $S = 100$).
2. When the chunk $\tilde{A}$ arrives 5 ticks later, the client sees its current step is 105.
3. It calculates the **staleness offset**: $Offset = 105 - 100 = 5$.
4. The client knows that index $0, 1, 2, 3, 4$ of the new chunk $\tilde{A}$ actually correspond to the time it was waiting. It aligns index 5 of the new chunk with the very next physical command to be sent.

6.2 The Aggregation Formula (f)

In the **smolVLA** framework (built on LeRobot), the aggregation function $f(A, \tilde{A})$ is typically a form of **Temporal Ensembling** (as cited from Zhao et al. 2023).

Instead of a simple average, the system calculates the command for each timestep t as a **weighted sum**:

$$a_t = \frac{\sum_i w_i \cdot a_{t,i}}{\sum_i w_i}$$

where $a_{t,i}$ is the prediction for time t from chunk i, and w_i is a weight that decays the further in the past the chunk was generated:

$$w_i = \exp(-m \cdot k)$$

where k is the "age" of the chunk in steps. This ensures that the robot prioritizes the *newest* perceptual information while still maintaining the *temporal smoothness* of the older prediction to prevent sudden jerks in motor voltage.

7 Summary

The asynchronous execution stack unlocks robust real-world deployments for VLA models. By setting a smart threshold g and utilizing a similarity filter, **smolVLA** achieves the responsiveness of continuous action evaluation while keeping computational costs and latency manageable.